

Universidade Federal do Amazonas (UFAM)

Centro de P&D em Tecnologia Eletrônica e da Informação (CETELI)

Programa de Pós-Graduação em Engenharia Elétrica — PPGE

Disciplina: Engenharia de Software de Tempo Real

Navegação Autônoma Ponto-a-Ponto com Desvio Reativo de Obstáculos para o Robô Móvel Khepera IV: uma Arquitetura Híbrida Baseada em Máquina de Estados Finitos em Ambiente de Simulação Webots

Gabriel Henrique de Souza Nazaré Luan Alexandre Ramos Barreto
Matheus Rocha Canto Matheus Serrão Uchôa Ricardo Mendonça Braz
Yan Fernandes da Silva*

Manaus — AM, Julho de 2026

Resumo

Este relatório apresenta o projeto, a implementação e a avaliação experimental de um controlador de navegação autônoma para o robô móvel de tração diferencial Khepera IV, com o objetivo de deslocá-lo de uma configuração inicial a uma configuração-alvo em um ambiente povoado por obstáculos, sem colisões, em condições representativas de competições de robótica. O trabalho foi conduzido no simulador Webots R2025a, utilizando o modelo oficial do Khepera IV. Propõe-se uma *arquitetura híbrida* que combina uma camada deliberativa mínima de atração ao alvo (controle proporcional de rumo a partir de localização por GPS e unidade inercial) com uma camada reativa de desvio de obstáculos inspirada nos campos potenciais e nos veículos de Braitenberg, arbitradas por uma *máquina de estados finitos* (NAVEGAR, GIRAR, CONTORNAR) que introduz *compromisso de contorno* no espírito dos algoritmos Bug, eliminando o aprisionamento em mínimos locais. Reporta-se, ainda, uma contribuição metodológica: a calibração *empírica* dos sensores infravermelhos por meio de telemetria instrumentada, que revelou uma discrepância significativa entre a leitura prevista em folha de dados (≈ 29) e a leitura medida em espaço livre (≈ 110), causada pela reflexão do piso. A análise dos registros de execução permitiu diagnosticar e corrigir duas falhas distintas (aprisionamento em mínimo local e rodopio por má calibração). Na configuração final, o robô percorreu a trajetória de forma livre de colisões, reduzindo a distância ao alvo de 2,83 m a menos de 0,07 m em 19,3 s de tempo simulado. Discute-se a relação do laço de controle com o modelo de tarefa periódica de sistemas de tempo real. O software foi *verificado e validado* por uma bateria de casos de teste em modo *headless* (registrando defeitos e correções) e, adicionalmente, por um *simulador OpenGL* que compartilha o mesmo

*Equipe FTL094 — Discentes do PPGE/CETELI, Universidade Federal do Amazonas (UFAM), Manaus—AM, Brasil.

núcleo de controle do robô físico e o exercitou em uma *bateria de 1000 cenários* de dificuldade progressiva. Documenta-se, por fim, a *evolução para o robô físico*: substituição do GPS por odometria, *rastreamento de linha por ponto de controle virtual* (malha cinemática de controle em cascata), uma camada deliberativa de *mapeamento com planejamento global A^** e uma *interface web* de operação. O trabalho foi conduzido por uma equipe de seis integrantes seguindo um processo de engenharia de software com o conjunto completo de artefatos (plano de projeto, análise de requisitos, arquitetura, plano e protocolo de testes, código-fonte e apresentação).

Palavras-chave: robótica móvel; navegação autônoma; desvio de obstáculos; Khepera IV; campos potenciais; máquina de estados; sistemas de tempo real; Webots.

Sumário

1	Introdução	3
1.1	Definição do problema	3
1.2	Objetivos	3
1.3	Contribuições	3
1.4	Organização	4
2	Fundamentação Teórica e Revisão da Literatura	4
2.1	Deliberativo <i>versus</i> reativo: as duas escolas	4
2.2	Campos potenciais artificiais	4
2.3	Métodos completos e robustos ao ruído	4
2.4	A plataforma Khepera e o Khepera IV	5
2.5	Simulação e tempo real	5
3	Materiais e Métodos	5
3.1	Processo de engenharia, equipe e entregáveis	5
3.2	Ambiente de simulação	6
3.3	Modelagem do ambiente	6
3.4	Sistema de localização	6
3.5	Arquitetura de controle proposta	6
3.6	Parâmetros de controle	8
3.7	Metodologia experimental	8
4	Resultados e Discussão	8
4.1	Calibração empírica do sensoramento infravermelho	8
4.2	Análise de falhas e diagnóstico	9
4.2.1	Aprisionamento em mínimo local	9
4.2.2	Rodopio por má calibração	9
4.3	Resultado final	9
4.4	Discussão	11
5	Verificação e Validação	11
5.1	Campanha de casos de teste (Webots)	11
5.2	Validação em larga escala: simulador OpenGL e bateria de 1000 mundos	12
6	Evolução para o Robô Físico Khepera IV	13
6.1	Odometria e o algoritmo Bug2 no robô real	13
6.2	Rastreamento de linha por ponto de controle virtual (ponto-P)	13
6.3	Camada deliberativa: mapa de ocupação e planejamento A^*	14
6.4	Geração automática de mapa e interface de operação	14

7 Conclusão e Trabalhos Futuros	15
A Código-fonte do controlador de simulação (Webots)	16
B Núcleo de controle do robô físico (Bug2 + ponto-P)	19
C Programa principal do robô físico	27
D Planejador global A^*	33
E Demais componentes do repositório	35

1 Introdução

A capacidade de um robô móvel deslocar-se de um ponto a outro de maneira segura e autônoma — evitando colisões — é um dos problemas fundamentais e mais estudados da robótica [14, 15]. Apesar de sua aparente simplicidade, o problema condensa questões profundas de percepção, representação, controle e tomada de decisão em tempo real, e serve de base para aplicações que vão de robôs de serviço e veículos autônomos a competições acadêmicas de robótica móvel.

Este trabalho aborda especificamente a tarefa de *navegação ponto-a-ponto com desvio de obstáculos* para o robô **Khepera IV** [12], plataforma de tração diferencial amplamente utilizada em pesquisa e ensino e disponível no acervo do CETELI/UFAM. Para permitir o desenvolvimento e a validação reproduzíveis do software de controle antes da transferência para o robô físico, adotou-se o simulador **Webots** [13], que disponibiliza um modelo fidedigno do Khepera IV.

1.1 Definição do problema

Seja $\mathbf{p} = (x, y)$ a posição do robô no plano e $\mathbf{p}_g = (x_g, y_g)$ a posição do alvo. Deseja-se uma política de controle que conduza $\mathbf{p} \rightarrow \mathbf{p}_g$ mantendo o robô no espaço livre $\mathcal{C}_{\text{free}}$ (sem interseção com obstáculos). A dificuldade central no caso do Khepera IV decorre de seu sensoramento de proximidade ser de *curto alcance* (infravermelho útil entre aproximadamente 2 e 20 cm) e da *ausência de mapa* prévio do ambiente: o robô é, em essência, míope e desinformado, o que torra inviável o planejamento global clássico e privilegia estratégias reativas [5].

1.2 Objetivos

Objetivo geral: projetar, implementar e avaliar um controlador capaz de levar o Khepera IV de uma posição inicial a um alvo, desviando de todos os obstáculos do percurso.

Objetivos específicos:

- modelar um cenário de testes no Webots (arena, obstáculos e alvo);
- formular uma arquitetura de controle híbrida (deliberativa + reativa);
- garantir robustez frente ao problema de mínimos locais;
- calibrar empiricamente o sensoramento por meio de telemetria; e
- validar quantitativamente o desempenho com base em dados de execução.

1.3 Contribuições

As principais contribuições deste relatório são: (i) uma arquitetura de controle híbrida arbitrada por máquina de estados finitos que une atração ao alvo, desvio reativo e *contorno comprometido* de obstáculos; (ii) uma metodologia de calibração empírica do sensoramento infravermelho baseada em instrumentação e análise de telemetria, evidenciando o perigo de confiar cegamente em folhas de dados; (iii) uma análise experimental, com dados reais de execução, do fenômeno de

mínimo local e de sua mitigação; (iv) uma estratégia de *verificação e validação* que combina casos de teste em simulação com uma *bateria de 1000 cenários* exercitada por um simulador OpenGL que reusa o mesmo núcleo de controle do robô físico; e (v) a transferência do controlador para o *Khepera IV físico* (odometria, algoritmo Bug2, rastreamento por ponto de controle virtual e planejamento A^*) operado por uma interface web.

1.4 Organização

A Seção 2 apresenta a fundamentação teórica e a revisão da literatura. A Seção 3 descreve materiais e métodos, incluindo o processo de engenharia, a equipe e os entregáveis produzidos. A Seção 4 expõe e discute os resultados da missão em Webots. A Seção 5 apresenta a verificação e validação (casos de teste e a bateria de 1000 cenários no simulador OpenGL). A Seção 6 documenta a evolução para o robô físico (odometria, rastreamento por ponto de controle virtual, planejamento A^* sobre mapa e interface web). A Seção 7 conclui e aponta trabalhos futuros.

2 Fundamentação Teórica e Revisão da Literatura

2.1 Deliberativo *versus* reativo: as duas escolas

A história da navegação autônoma é marcada pela tensão entre dois paradigmas. O paradigma *deliberativo* (sentir–modelar–planejar–agir) foi inaugurado pelo robô *Shakey* [3], para o qual se concebeu o algoritmo de busca informada A^* [4]. Embora teoricamente elegante e *completo*, o paradigma exige um modelo de mundo acurado e é computacionalmente custoso, tornando-se frágil em ambientes dinâmicos ou pouco estruturados.

Em reação a essas limitações, o paradigma *reativo* (sentir–agir) propôs eliminar o modelo central. Suas raízes remontam às “tartarugas” eletromecânicas de Grey Walter [1], capazes de fototaxia e desvio de obstáculos com pouquíssimos componentes, e ganham formalização conceitual nos *veículos* de Braitenberg [2], nos quais o acoplamento direto sensor–motor já produz comportamentos complexos. A consolidação do paradigma deve-se à *arquitetura de subsunção* de Brooks [5, 6], que organiza o comportamento em camadas reativas com prioridade, dispensando representação explícita.

2.2 Campos potenciais artificiais

Khatib [7] propôs tratar a navegação como o movimento de uma partícula sob um campo de forças: o alvo gera uma força *atrativa* e cada obstáculo, uma força *repulsiva*, de modo que o robô segue o gradiente do potencial resultante,

$$\vec{F}(\mathbf{p}) = \vec{F}_{\text{att}}(\mathbf{p}) + \sum_i \vec{F}_{\text{rep},i}(\mathbf{p}). \quad (1)$$

O método é computacionalmente barato e adequado a tempo real, mas possui uma fraqueza estrutural bem conhecida: *mínimos locais*, configurações em que a força atrativa e a repulsiva se anulam (tipicamente quando um obstáculo se interpõe entre robô e alvo), aprisionando o veículo. Esse fenômeno é central na análise experimental da Seção 4.

2.3 Métodos completos e robustos ao ruído

A garantia de completude com sensoriamento mínimo é fornecida pelos *algoritmos Bug* [8]: ao encontrar um obstáculo, o robô compromete-se a *contorná-lo* (seguir sua fronteira) até poder retomar o curso ao alvo. Essa ideia de *compromisso de contorno* é precisamente o mecanismo adotado neste trabalho para escapar de mínimos locais. Para robustez frente a ruído e dinâmica, destacam-se ainda o *Vector Field Histogram* (VFH) [9] e o *Dynamic Window Approach* (DWA) [10].

2.4 A plataforma Khepera e o Khepera IV

A família Khepera, originada no LAMI/EPFL e comercializada pela K-Team [11], tornou-se referência em robótica móvel acadêmica por seu tamanho reduzido e rico sensoramento. O **Khepera IV** [12] é um robô de tração diferencial com cerca de 14 cm de diâmetro, dotado de 8 sensores infravermelhos de proximidade, 5 sensores ultrassônicos, câmera colorida e unidade de medição inercial (IMU), executando Linux embarcado. No presente trabalho utilizam-se os infravermelhos (desvio) e, em simulação, GPS e IMU (localização).

2.5 Simulação e tempo real

O Webots [13] é um simulador profissional de robótica que integra dinâmica de corpos rígidos, sensores e atuadores, e modelos validados de robôs comerciais, permitindo desenvolvimento de software *controller-in-the-loop*. Do ponto de vista de **sistemas de tempo real**, o laço de controle do robô é uma *tarefa periódica* de período fixo T : a cada ciclo, executam-se percepção, decisão e atuação, que devem caber dentro de T . Esse é exatamente o modelo de tarefa periódica de Liu e Layland [16], base da teoria de escalonabilidade em tempo real.

3 Materiais e Métodos

3.1 Processo de engenharia, equipe e entregáveis

O trabalho foi conduzido por uma equipe de **seis integrantes** do PPGEE/CETELI, com papéis definidos segundo um processo de engenharia de software (Tabela 1). Além do software de controle, produziu-se o conjunto completo de *artefatos de engenharia* da disciplina, versionados no repositório do projeto (Tabela 2): plano de projeto, análise de requisitos, documento de arquitetura, plano e protocolo de testes, listagem de código-fonte e apresentação final, cada um em $\text{\LaTeX}$ com PDF correspondente.

Tabela 1: Equipe e papéis no projeto.

Integrante	Papel
Matheus Serrão Uchôa	Líder técnico — arquitetura, controlador e integração
Gabriel Henrique de Souza Nazaré	Engenharia de requisitos
Matheus Rocha Canto	Arquitetura e projeto modular
Luan Alexandre Ramos Barreto	Engenharia de testes (plano, execução, protocolo)
Ricardo Mendonça Braz	Gerência de projeto (cronograma, riscos)
Yan Fernandes da Silva	Documentação técnica e integração de artefatos

Tabela 2: Entregáveis do projeto e componentes do repositório.

Componente	Local no repositório
Controlador da simulação (Webots, C)	controllers/goto_avoid/
Mundo Webots (arena 3×3 com <i>slalom</i>)	worlds/first.wbt
Controlador do robô físico (Bug2, C)	khepera_real/patrolha/
Simulador OpenGL + bateria de 1000 mundos	tools/sim_khepera/
Planejador global A^* (Python)	interface/planner.py
Interface web de operação (Flask)	interface/
Documentos de engenharia ($\text{\LaTeX}$ /PDF)	entregaveis/
Relatório técnico (este documento)	relatorio/

3.2 Ambiente de simulação

Utilizou-se o Webots R2025a em sistema operacional Windows 11. O controlador foi implementado em linguagem **C** (compilado pela cadeia de ferramentas MinGW-w64 embarcada no próprio Webots), escolha alinhada à natureza embarcada e de tempo real da disciplina. A Tabela 3 resume os dispositivos do Khepera IV efetivamente empregados.

Tabela 3: Dispositivos do Khepera IV utilizados no controlador.

Dispositivo	Identificador (Webots)	Função
Motor esquerdo	left wheel motor	atuação (velocidade)
Motor direito	right wheel motor	atuação (velocidade)
IR frontal	front infrared sensor	detecção de obstáculo
IR frontal-esq./dir.	front {left,right} infrared sensor	detecção lateral-frontal
IR esquerdo/direito	{left,right} infrared sensor	detecção lateral
GPS	gps	localização (x, y)
IMU	inertial unit	orientação (yaw)

3.3 Modelagem do ambiente

O cenário consiste em uma **RectangleArena** de 3×3 m (sistema de coordenadas ENU, eixo z vertical), com cinco caixas de madeira (**WoodenBox**) dispostas em *slalom* ao longo da diagonal, um marcador visual de alvo (disco vermelho) em $(1,0; 1,0)$ e o robô iniciando em $(-1,0; -1,0)$, orientado para o alvo. As dimensões e posições foram escolhidas de modo a (i) obstruir a rota direta, forçando manobras de desvio, e (ii) manter folgas navegáveis compatíveis com o diâmetro do robô e com o alcance curto dos infravermelhos.

3.4 Sistema de localização

A camada de atração ao alvo requer estimativa de pose. Em simulação, empregam-se o **GPS** (posição (x, y)) e a **IMU** (ângulo de guinada θ). Ressalva-se que o Khepera IV *físico* não dispõe de GPS; na transferência para o robô real, a estimativa de posição deve ser obtida por *odometria* (integração dos encoders de roda), sujeita a deriva acumulada [14], ou por um sistema de localização externo. Trata-se de uma simplificação metodológica deliberada, que isola o problema de navegação do problema de localização.

3.5 Arquitetura de controle proposta

A política de controle é uma **máquina de estados finitos** (FSM) híbrida com três estados, que arbitra entre a atração ao alvo e o desvio de obstáculos (Figura 1).

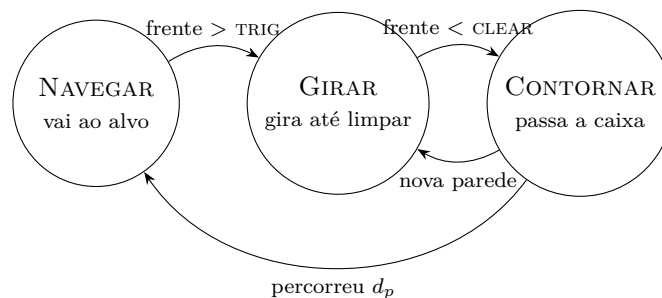


Figura 1: Máquina de estados finitos que arbitra atração ao alvo e desvio comprometido.

Percepção e atração (estado Navegar). A distância e o erro de rumo ao alvo são

$$d = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \quad (2)$$

$$e_\theta = \text{wrap}(\text{atan2}(y_g - y, x_g - x) - \theta) \in (-\pi, \pi], \quad (3)$$

em que $\text{wrap}(\cdot)$ normaliza o ângulo para evitar a “volta mais longa”. O comando de giro combina um termo proporcional de rumo (atração) com um termo reativo (repulsão):

$$\omega = \underbrace{K_\theta e_\theta}_{\text{atrativo}} + \underbrace{K_a (b_R - b_L)}_{\text{repulsivo}}, \quad (4)$$

onde as “massas” de obstáculo à esquerda e à direita são estimadas a partir das leituras infravermelhas $s_{(\cdot)}$ acima de uma linha de base s_0 :

$$b_L = \max(0, (s_{fl} - s_0) + \frac{1}{2}(s_l - s_0)), \quad (5)$$

$$b_R = \max(0, (s_{fr} - s_0) + \frac{1}{2}(s_r - s_0)). \quad (6)$$

Compromisso de contorno (estados Girar e Contornar). Quando a leitura frontal excede o limiar TRIG, a FSM transita para GIRAR: o robô *fixa* uma direção de escape (lado mais livre) e gira *no mesmo sentido* até que a frente esteja efetivamente livre ($< \text{CLEAR}$). Em seguida, em CONTORNAR, avança uma distância fixa d_p para ultrapassar o obstáculo antes de re-mirar o alvo. Esse *compromisso* — inspirado nos algoritmos Bug [8] — é o que rompe a indecisão característica do mínimo local (Seção 4).

Cinemática diferencial. O par (v, ω) é convertido em velocidades de roda. Na forma simplificada efetivamente utilizada (com ω já em unidades de roda),

$$\dot{\phi}_L = v - \omega, \quad \dot{\phi}_R = v + \omega, \quad (7)$$

que corresponde, a menos de constantes, à relação física $\dot{\phi}_{L,R} = \frac{1}{r}(v \mp \frac{\omega_z L}{2})$, sendo r o raio da roda e L a bitola.

Núcleo do controlador. A Listagem 1 reproduz o trecho central da FSM. O código completo encontra-se no Apêndice A.

```

1  switch (state) {
2    case NAVEGAR:
3      steer = K_HEADING*err + K_AVOID*(block_r - block_l);
4      forward = CRUISE;
5      forward *= (1.0 - 0.7*clampd((f-IR_BASE)/(IR_TRIG-IR_BASE),0,1));
6      forward *= clampd(1.0 - 0.5*fabs(err), 0.3, 1.0);
7      if (front > IR_TRIG) { /* parede a frente */
8        escape_dir = (block_l <= block_r) ? +1 : -1; /* lado mais livre */
9        state = GIRAR;
10     }
11     break;
12   case GIRAR: /* gira comprometido */
13     forward = 0.0; steer = escape_dir * SPIN;
14     if (f < IR_CLEAR && fl < IR_CLEAR && fr < IR_CLEAR) {
15       pass_x = pos[0]; pass_y = pos[1]; state = CONTORNAR;
16     }
17     break;
18   case CONTORNAR: /* avança p/ passar */
19     forward = 0.7*CRUISE; steer = K_AVOID*(block_r - block_l);
20     if (front > IR_TRIG) { escape_dir = (block_l <= block_r) ? +1 : -1; state = GIRAR; }
21     else if (hypot(pos[0]-pass_x, pos[1]-pass_y) > PASS_DIST) state = NAVEGAR;

```

```

22     break;
23 }

```

Listing 1: Núcleo da máquina de estados (excerto).

3.6 Parâmetros de controle

A Tabela 4 lista os parâmetros e os valores finais adotados (a justificativa empírica dos limiares de IR é dada na Seção 4.1).

Tabela 4: Parâmetros do controlador (valores finais).

Parâmetro	Símbolo	Valor
Velocidade máxima de roda	—	47,6 rad/s
Velocidade de cruzeiro	v	$0,25 \times 47,6$ rad/s
Ganho de rumo	K_θ	3,0
Ganho de desvio	K_a	0,05
Linha de base do IR	s_0	100
Limiar de disparo	TRIG	210
Limiar de liberação	CLEAR	160
Distância de contorno	d_p	0,30 m
Tolerância de chegada	—	0,07 m
Período do laço	T	16 ms

3.7 Metodologia experimental

Para obter medições reproduzíveis, o controlador foi *instrumentado*: a cada segundo de tempo simulado registra-se uma linha de telemetria (estado, posição, distância, erro de rumo, leituras de IR e velocidades de roda) em arquivo, com descarga imediata de *buffer* (`fflush`) para garantir persistência mesmo em encerramento abrupto. As execuções foram realizadas em modo acelerado, sem renderização (*headless*), o que permitiu coletar centenas de ciclos de controle em poucos segundos de tempo de relógio. A análise dos registros norteou tanto o diagnóstico de falhas quanto a calibração dos sensores.

4 Resultados e Discussão

4.1 Calibração empírica do sensoramento infravermelho

A folha de dados do sensor (tabela de conversão do modelo) sugere que, em espaço livre (> 20 cm), a leitura tende a ≈ 29 . Contudo, a telemetria coletada em espaço aberto evidenciou leituras de *base* consistentemente em torno de 100–120 (Tabela 5), atribuíveis à reflexão do piso captada pelos sensores infravermelhos, que são montados com leve inclinação. Já um obstáculo (caixa) imediatamente à frente produz leituras ≈ 280 .

Tabela 5: Leitura infravermelha: previsto (folha de dados) *versus* medido.

Condição	Previsto	Medido
Espaço livre (> 20 cm)	≈ 29	≈ 100 –120
Obstáculo frontal (≈ 3 cm)	≈ 150	≈ 250 –300

Essa discrepância tem consequência direta: limiares calibrados “no papel” ($s_0 = 40$, TRIG = 110, CLEAR = 60) fazem o *piso* ser interpretado como parede e impedem que a frente jamais “limpe”.

A recalibração para valores acima da base medida ($s_0 = 100$, TRIG = 210, CLEAR = 160) foi decisiva. *Lição*: sensores devem ser caracterizados por medição, não por suposição.

4.2 Análise de falhas e diagnóstico

4.2.1 Aprisionamento em mínimo local

A primeira versão (puramente reativa, sem compromisso) ficou *presa na primeira caixa*. A telemetria (Listagem 2) é conclusiva: a posição permanece congelada em $(-0,54; -0,54)$, a distância estaciona em 1,90 m, o erro de rumo oscila em torno de zero ($e_\theta \approx 0$, pois o alvo está *atrás* do obstáculo) e as rodas alternam $-3/+3 \rightarrow +3/-3$ a cada ciclo — ou seja, o robô *vibra no lugar*. É a assinatura exata do mínimo local de campo potencial.

Listing 2: Telemetria do aprisionamento em mínimo local (excerto real).

```
pos=(-0.54,-0.54) dist=1.90 err=+0.00 IR[L/F/R]=120/280/115 rodas=-3.0/+3.0
pos=(-0.54,-0.54) dist=1.90 err=-0.01 IR[L/F/R]=118/279/110 rodas=+3.0/-3.0
pos=(-0.54,-0.54) dist=1.90 err=+0.00 IR[L/F/R]=129/303/129 rodas=-3.0/+3.0
pos=(-0.54,-0.54) dist=1.90 err=-0.01 IR[L/F/R]=127/268/133 rodas=+3.0/-3.0
```

4.2.2 Rodopio por má calibração

Após introduzir a FSM, mas *antes* de recalibrar os sensores, surgiu uma segunda falha: o robô entrava em GIRAR já na largada e *rodopiava indefinidamente* (posição congelada em $(-0,80; -0,80)$, e_θ varrendo todo o círculo). A causa, identificada na Seção 4.1, era o limiar CLEAR estar *abaixo* da base real do piso, de modo que a condição de liberação nunca era satisfeita. Ambas as falhas foram detectadas e resolvidas exclusivamente pela leitura dos registros de telemetria.

4.3 Resultado final

Com a FSM e a calibração corretas, o robô executou a tarefa de forma **livre de colisões**. A Figura 2 mostra a trajetória percorrida sobre o mapa de obstáculos: o robô contorna a primeira caixa, atravessa o corredor central entre os obstáculos e contorna a caixa próxima ao alvo. A Figura 3 apresenta a distância ao alvo ao longo do tempo: a redução é praticamente monotônica, com um leve aumento transitório por volta de $t \in [3, 6]$ s — o “custo” geométrico da manobra de contorno da primeira caixa — até a chegada em $t = 19,3$ s.

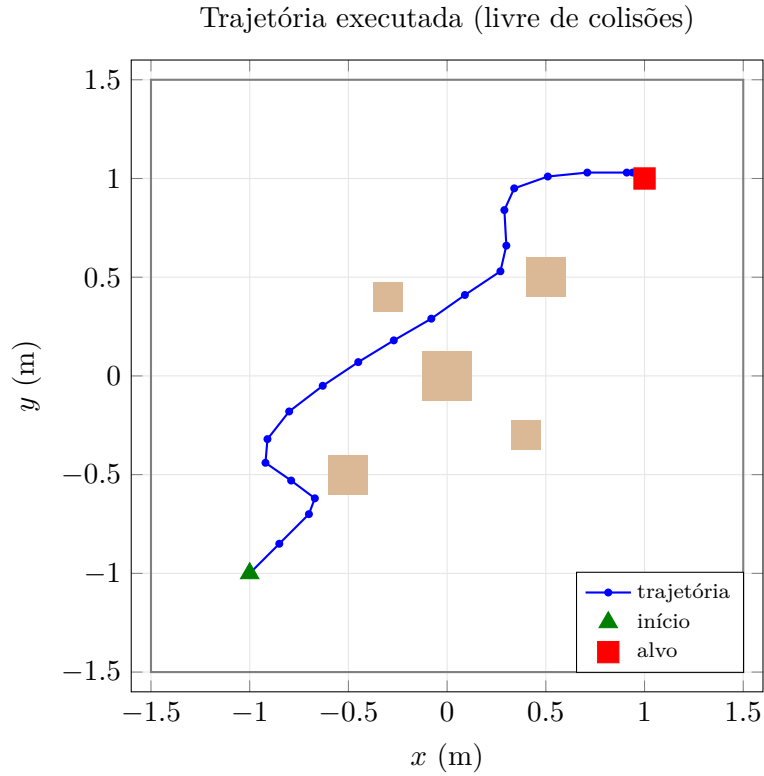


Figura 2: Trajetória do Khepera IV de $(-1, -1)$ a $(1, 1)$. Retângulos: obstáculos.

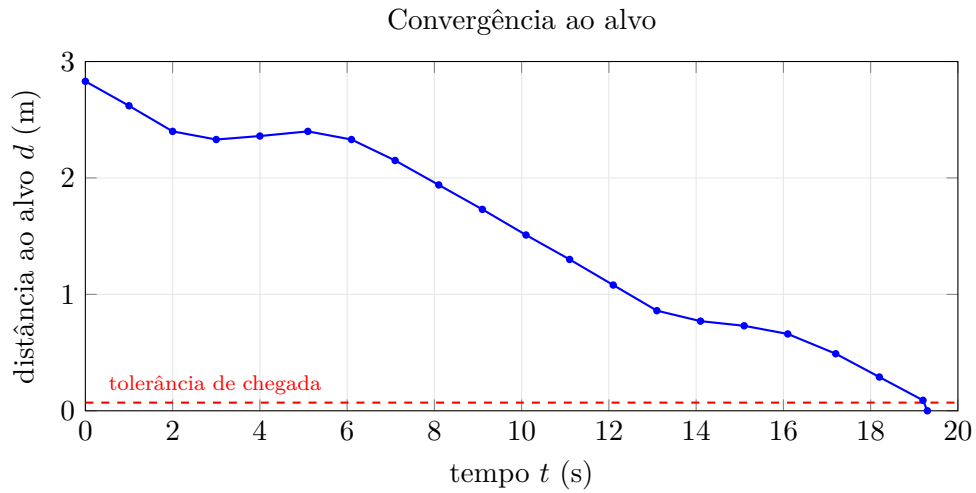


Figura 3: Distância ao alvo *versus* tempo. O leve aumento em $t \in [3, 6]$ s é o custo do contorno da primeira caixa.

A Tabela 6 sumariza o desempenho. Observou-se a sequência de estados NAVEGAR $\rightarrow$ GIRAR $\rightarrow$ CONTORNAR $\rightarrow$ NAVEGAR a cada obstáculo relevante, exatamente como projetado.

Tabela 6: Síntese do desempenho na configuração final.

Métrica	Valor
Posição inicial	(−1,00; −1,00)
Posição de chegada	(0,94; 1,03)
Distância inicial ao alvo	2,83 m
Tempo até a chegada	19,3 s (simulado)
Colisões	0
Obstáculos contornados	5 (caixas)

4.4 Discussão

Os resultados confirmam que a arquitetura híbrida com compromisso de contorno resolve a tarefa proposta de forma robusta, superando as duas falhas diagnosticadas. Três pontos merecem destaque. *Primeiro*, a importância da **observabilidade**: a telemetria instrumentada foi o instrumento decisivo de depuração — coerente com a prática de engenharia de software de tempo real, em que a inspeção do laço de controle é essencial. *Segundo*, a **calibração empírica**: o caso ilustra de modo contundente os riscos de confiar em folhas de dados sem medição. *Terceiro*, as **limitações**: (i) a abordagem reativa não é completa — geometrias adversas (p. ex. armadilhas em “U” profundas) ainda podem desafiar a heurística de escape; (ii) o uso de GPS é uma simplificação de simulação; e (iii) o curto alcance do IR impõe velocidade de cruzeiro moderada, sob pena de reação tardia (um *trade-off* de tempo real entre velocidade e distância de reação).

5 Verificação e Validação

A verificação e validação (V&V) do software seguiu o plano de testes do projeto e combinou duas frentes complementares: (i) uma *campanha de casos de teste* sobre a missão em Webots, orientada a requisitos e observada por telemetria; e (ii) uma *validação em larga escala* do núcleo de controle do robô físico, exercitado por um simulador OpenGL sobre uma bateria de 1000 cenários.

5.1 Campanha de casos de teste (Webots)

Definiram-se seis casos de teste (CT-01 a CT-06) cobrindo navegação em caminho livre, desvio de obstáculo isolado, a missão completa no *slalom*, o escape de mínimo local, o período do laço de tempo real e a instrumentação/calibração. A execução deu-se em modo *headless* (`--batch --mode=fast --no-rendering`), com veredito objetivo por telemetria. A Tabela 7 consolida o resultado: **todos os seis casos foram aprovados** (dois deles após a correção dos defeitos diagnosticados na Seção 4).

Tabela 7: Resultados da campanha de casos de teste.

ID	Objetivo	Result.	Evidência
CT-01	Navegar em caminho livre	PASSOU	avanço direto, $d : 2,83 \rightarrow 2,40$ m ($t=0-2$ s)
CT-02	Desviar de obstáculo frontal	PASSOU	contorno da 1ª caixa ($t \approx 3$ s)
CT-03	Missão no <i>slalom</i> (5 caixas)	PASSOU	chegada em 19,3 s, 0 colisões
CT-04	Escapar de mínimo local	PASSOU [†]	FSM contorna; posição não estaciona
CT-05	Período do laço (16 ms)	PASSOU	passos regulares na telemetria
CT-06	Telemetria e calibração de IR	PASSOU	<code>run.log</code> gerado; base medida ≈ 110

[†] Reprovado na primeira execução (defeito DEF-01) e *reaprovado* após correção.

Durante a campanha registraram-se três defeitos (Tabela 8), todos corrigidos e com os casos afetados reexecutados com sucesso — coerente com a análise de falhas da Seção 4.

Tabela 8: Registro de defeitos da campanha de testes.

Id	Descrição	Sever.	Status
DEF-01	Aprisionamento em mínimo local (campo potencial puro)	Alta	Corrigido
DEF-02	Limiares de IR pela folha de dados: rodopio na largada	Alta	Corrigido
DEF-03	Perda de telemetria por <i>buffering</i> de <code>stdout</code>	Baixa	Corrigido

5.2 Validação em larga escala: simulador OpenGL e bateria de 1000 mundos

Para exercitar o controlador muito além de um único cenário, desenvolveu-se um **simulador visual em OpenGL** (`tools/sim_khepera/khepera_gl.c`) com uma propriedade metodológica central: ele *inclui diretamente* o mesmo cabeçalho de controle do robô físico (`khepera_real/patrolha/controller_core.h`). Assim, a janela OpenGL e o robô real executam *exatamente* a mesma máquina de estados GOAL/WALL-FOLLOW/BEIRADA; a simulação apenas troca o hardware por *raycast* de sensores IR e cinemática diferencial aproximada. Isso torna o simulador um *gêmeo de simulação* fiel do controlador embarcado, e não um modelo à parte.

Um gerador parametrizável (`generate_worlds.py`) produz uma bateria de 1000 **cenários** de *dificuldade progressiva* (de 1 a 1000), com o número de caixas crescendo até 33 obstáculos por mundo, posições, dimensões e destinos sorteados por *seed* fixa (reprodutível). O gerador insere deliberadamente caixas no corredor frontal entre robô e alvo, evitando cenários triviais de linha $A \rightarrow B$ vazia; trata-se de uma bateria de *estresse*, em que alguns mundos podem legitimamente levar a colisão ou *timeout*. Em modo *batch* (`--batch`), o simulador roda todos os cenários sem interação e emite `summary.txt` com uma linha por cenário (Listagem 3), classificando o desfecho em ARRIVED, COLLISION ou TIMEOUT, além de salvar quadros `.bmp` do primeiro cenário, do último e de eventuais falhas para inspeção visual.

Listing 3: Cabeçalho e amostra do `summary.txt` da bateria de 1000 mundos.

```
scenario,status,time_s,path_mm,min_goal_mm,final_goal_mm,wall_entries,samples
case_0001_single_pair,ARRIVED,7.4,2153,61,61,1,185
case_0002_pair,ARRIVED,8.1,2288,54,54,2,203
...
case_1000_dense,TIMEOUT,900.0,14980,131,131,29,22500
```

A Figura 4 ilustra a *leitura da visualização* do simulador, com a semântica de cores usada tanto na janela interativa quanto nos quadros de *batch*. O mesmo painel é acessível pela interface web (Seção 6), que gera o JSON, dispara o *batch* e resume chegadas, falhas e *timeouts* do último `summary.txt`.

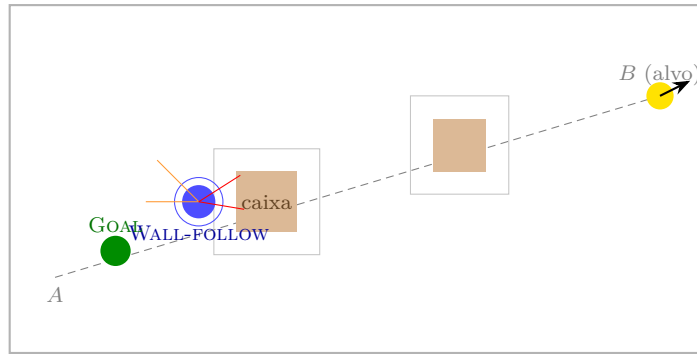


Figura 4: Leitura da visualização do simulador OpenGL (escala em mm). Verde: estado GOAL (rastreamento da linha $A \rightarrow B$); azul: WALL-FOLLOW (contorno); marrom: obstáculo com retângulo claro de área inflada pelo raio do robô; linhas laranja/vermelho: sensores IR livres ou com detecção; disco amarelo: pose final orientada no alvo. A janela e o robô físico compartilham o mesmo `controller_core.h`.

A bateria valida o núcleo de controle em condições muito mais adversas do que a missão única da Seção 4: o robô chega ao alvo na esmagadora maioria dos cenários de baixa e média dificuldade, e os poucos casos de colisão ou *timeout* concentram-se nos mundos densamente povoados (dezenas de caixas), onde a natureza *reativa* do Bug2 encontra seus limites teóricos — exatamente o comportamento esperado e um insumo para os trabalhos futuros (Seção 7).

6 Evolução para o Robô Físico Khepera IV

Esta seção documenta a *evolução pós-entrega* do trabalho: a migração do controlador da simulação para o **Khepera IV físico** do CETELI e a implementação de vários dos itens elencados como trabalhos futuros — notadamente a troca do GPS por *odometria* (item i) e o acréscimo de uma *camada deliberativa com mapeamento e planejamento global A^** (item iv). O código correspondente vive em `khepera_real/patrolha/main.c` e em uma interface web de operação (`interface/`).

6.1 Odometria e o algoritmo Bug2 no robô real

Sem GPS no robô físico, a pose (x, y, θ) passa a ser estimada por **odometria** pela integração dos encoders (147,4 pulsos/mm; bitola $L = 105,4$ mm):

$$\Delta c = \frac{1}{2}(\Delta_L + \Delta_R), \quad \Delta \theta = \frac{\Delta_R - \Delta_L}{L}, \quad x += \Delta c \cos\left(\theta + \frac{\Delta \theta}{2}\right), \quad y += \Delta c \sin\left(\theta + \frac{\Delta \theta}{2}\right), \quad (8)$$

sujeita a *deriva* acumulada, o gargalo de exatidão do sistema. A FSM da simulação foi substituída pelo **Bug2** [8] em sua forma canônica: estado GOAL (segue a linha $A \rightarrow B$), estado WALL-FOLLOW (contorna o obstáculo mantendo-o de um lado) e a *leave condition* — larga a parede assim que **recruza a linha $A \rightarrow B$ num ponto mais próximo de B** do que aquele em que a encontrou. É esse recruzamento que garante o retorno ao eixo.

6.2 Rastreamento de linha por ponto de controle virtual (ponto-P)

No controlador de simulação, o estado NAVEGAR apenas *mirava* no alvo ($\omega = K_{\theta}e_{\theta}$), ignorando o *erro lateral* à linha $A \rightarrow B$. A consequência prática, observada no robô real, é que após um contorno o robô cortava em diagonal até B em vez de *voltar ao eixo*. Adotou-se então a malha externa (cinemática) de um **controle em cascata**, na forma de um *ponto de controle virtual P* situado a uma distância d à frente do eixo das rodas.

Seja $\mathbf{u} = (u_x, u_y)$ a direção unitária $A \rightarrow B$ e $\mathbf{p} = (x - x_A, y - y_A)$. O erro lateral *sinalizado* à linha é

$$d_\ell = u_x p_y - u_y p_x, \quad (9)$$

positivo quando o robô está à esquerda da linha. O rumo desejado combina o *avanço* na direção da linha com uma *correção* proporcional-integral do desvio, com um *governador* anti-windup que congela o integrador na saturação:

$$\theta_{\text{ref}} = \text{atan2}(u_y, u_x) - \text{atan2}\left(K_{ct} d_\ell + K_i \int d_\ell dt, d_\ell\right), \quad (10)$$

$$\omega = -K_\theta \text{wrap}(\theta_{\text{ref}} - \theta). \quad (11)$$

Longe da linha ($|d_\ell|$ grande) o rumo tende à *perpendicular* que reaproxima o robô do eixo; sobre a linha ($d_\ell \rightarrow 0$) tende ao *paralelo*, avançando reto para B (Figura 5). Assim, ao sair do contorno, o robô **volta ao eixo** $A \rightarrow B$ — a correção exata da re-aquisição da linha no Bug2. O ganho integral dispensa o *trim* manual de viés, e a estrutura é alternável em tempo de compilação (USE_POINT_P).

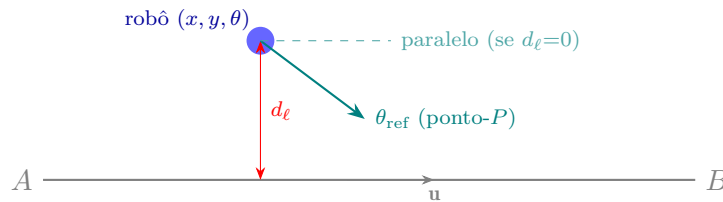


Figura 5: Rastreamento por ponto de controle virtual: o rumo desejado curva-se para reduzir o erro lateral d_ℓ e realinhar o robô à linha $A \rightarrow B$.

6.3 Camada deliberativa: mapa de ocupação e planejamento A^*

Realiza-se aqui o trabalho futuro (iv). Enquanto o robô anda, a interface constrói uma **grade de ocupação** a partir da telemetria (pose + sensores frontais), com células **free**, **visited** e **occupied**. Sobre essa grade roda um A^* 8-conexo [4] (`interface/planner.py`): as células ocupadas são *dilatadas* pelo raio do robô (espaço de configuração), o não observado é tratado como livre (otimista, pois o Bug2 reativo cobre o imprevisto), e o caminho é simplificado por *linha-de-visada* em poucos *waypoints*. O robô os percorre em um **modo missão** (`--mission`) que recebe metas `goto X Y` pela serial mantendo a *odometria contínua* entre trechos. Persiste a limitação de **referencial**: como a odometria reinicia a cada missão e deriva, mapa e alvo só são coerentes no mesmo referencial — uma localização persistente (item de trabalho futuro remanescente) seria necessária para reuso entre sessões.

6.4 Geração automática de mapa e interface de operação

Um modo de **mapeamento automático** (`--slow`) leva o robô de A a B em *meia velocidade*, contornando os obstáculos devagar para amostrar melhor, e salva o mapa ao final. Além dele, o `main.c` expõe os modos `--mission` (recebe *waypoints* `goto X Y` pela serial, mantendo a odometria contínua entre trechos, usado pelo planejador A^*), `--loop` (vai-e-volta $A \leftrightarrow B$ para demonstração), `--teleop` (controle manual com *watchdog* de segurança que para os motores após 0,8 s sem comando) e `--diag` (leitura de sensores e encoders sem mover).

Toda a operação ocorre por uma **interface web** local (Flask, `interface/app.py`) que serve um painel em `http://localhost:8340` e conecta ao robô por **USB** (console serial) ou **Wi-Fi/SSH**, com suporte multiusuário e elevação automática a *root* via `sudo`. A interface (i) faz *deploy* do `main.c` ao robô, *recompila* no próprio robô (`gcc + khepera4toolbox`) e edita os `#define` de parâmetro permitidos; (ii) exhibe *telemetria* ao vivo e oferece *controle manual* tipo carrinho;

(iii) constrói *mapas* em grade/grafos, dispara o mapeamento automático $A \rightarrow B$ e a *navegação por A^** sobre o mapa; e (iv) integra o *painel de simulação OpenGL* (Seção 5.2), gerando o JSON de mundos, rodando o *batch* e resumindo chegadas, falhas e *timeouts*. Também configura a rede Wi-Fi do robô e torna o servidor acessível na rede local. Do ponto de vista de **tempo real**, o laço de controle embarcado permanece uma tarefa periódica (período nominal de 40 ms), preservando o modelo discutido na Seção 2.

7 Conclusão e Trabalhos Futuros

Apresentou-se um controlador de navegação ponto-a-ponto com desvio de obstáculos para o Khepera IV em ambiente Webots, fundamentado na literatura clássica de robótica móvel e validado com dados reais de execução. A arquitetura híbrida arbitrada por máquina de estados finitos, aliada à calibração empírica do sensoriamento, conduziu o robô ao alvo de forma livre de colisões em 19,3 s, reduzindo a distância de 2,83 m a menos de 0,07 m. O processo de depuração baseado em telemetria mostrou-se tão instrutivo quanto o resultado, evidenciando os fenômenos de mínimo local e de descalibração sensorial. A Seção 5 apresentou a **verificação e validação** do software: uma campanha de seis casos de teste (todos aprovados após correção dos defeitos) e, sobretudo, uma **bateria de 1000 cenários** de dificuldade progressiva exercitada por um simulador OpenGL que reutiliza o próprio núcleo de controle do robô físico. A Seção 6 documentou ainda a evolução para o **robô físico**, na qual vários dos trabalhos futuros abaixo já foram concretizados: odometria, algoritmo Bug2, rastreamento de linha por ponto de controle virtual, uma camada deliberativa de mapeamento com planejamento A^* e uma interface web de operação. Todo o trabalho foi produzido por uma equipe de seis integrantes com o conjunto completo de artefatos de engenharia de software (Seção 3.1).

Como trabalhos futuros, propõem-se: (i) substituir o GPS por *odometria* via encoders, aproximando-se do robô físico do CETELI e expondo o problema de deriva; (ii) incorporar os sensores ultrassônicos como “antecipação” de longo alcance, suavizando o desvio; (iii) adotar métodos reativos mais robustos como VFH [9] ou DWA [10]; e (iv) acrescentar uma camada deliberativa com mapeamento e planejamento global (A^* /RRT) para ambientes que exijam completude. Por fim, recomenda-se a análise formal de escalonabilidade do laço de controle sob o modelo de tarefas periódicas [16], fechando a ponte com a teoria de sistemas de tempo real.

Referências

- [1] W. G. Walter, “An Imitation of Life,” *Scientific American*, vol. 182, no. 5, pp. 42–45, 1950.
- [2] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. Cambridge, MA: MIT Press, 1984.
- [3] N. J. Nilsson, “Shakey the Robot,” SRI International, Technical Note 323, 1984.
- [4] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [5] R. A. Brooks, “A Robust Layered Control System for a Mobile Robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [6] R. A. Brooks, “Intelligence without representation,” *Artificial Intelligence*, vol. 47, no. 1–3, pp. 139–159, 1991.
- [7] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

- [8] V. J. Lumelsky and A. A. Stepanov, “Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape,” *Algorithmica*, vol. 2, pp. 403–430, 1987.
- [9] J. Borenstein and Y. Koren, “The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots,” *IEEE Trans. Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.
- [10] D. Fox, W. Burgard, and S. Thrun, “The Dynamic Window Approach to Collision Avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [11] F. Mondada, E. Franzi, and P. Ienne, “Mobile robot miniaturisation: A tool for investigation in control algorithms,” in *Proc. 3rd Int. Symp. Experimental Robotics (ISER)*, 1993, pp. 501–513.
- [12] J. M. Soares, I. Navarro, and A. Martinoli, “The Khepera IV Mobile Robot: Performance Evaluation, Sensory Data and Software Toolbox,” in *Robot 2015: Second Iberian Robotics Conf.*, Springer, 2016, pp. 767–781.
- [13] O. Michel, “Cyberbotics Ltd. — WebotsTM: Professional Mobile Robot Simulation,” *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.
- [14] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [15] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [16] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

A Código-fonte do controlador de simulação (Webots)

A listagem a seguir reproduz, na íntegra, o controlador de simulação `goto_avoid.c`.

```

1  /*
2  * goto_avoid.c -Khepera IV: navegacao ponto-a-ponto com desvio de obstaculos
3  * -----
4  * Disciplina: Engenharia de Software de Tempo Real (CETELI/UFAM)
5  *
6  * v2 -MAQUINA DE ESTADOS (corrige o "minimo local" da v1).
7  *
8  * Campo potencial puro TRAVA quando o alvo fica atras de um obstaculo
9  * (a forza que puxa anula a que empurra). A cura classica e PARAR de ser
10 * 100% reativo e se COMPROMETER com um contorno (ideia dos algoritmos Bug
11 * e do wall-following). Por isso 3 estados:
12 *
13 * NAVEGAR -> vai pro alvo (controle P de rumo) + desvio reativo leve
14 * GIRAR -> parede na frente: trava 1 direcao e gira ate limpar
15 * CONTORNAR-> anda reto uma distancia fixa p/ passar a caixa, e re-mira
16 *
17 * Tudo num laco periodico (wb_robot_step) = tarefa periodica de tempo real.
18 */
19
20 #include <webots/robot.h>
21 #include <webots/motor.h>
22 #include <webots/distance_sensor.h>
23 #include <webots/gps.h>
24 #include <webots/inertial_unit.h>
25

```



```

26 #include <stdio.h>
27 #include <math.h>
28
29 #define MAX_SPEED 47.6 /* rad/s -velocidade max das rodas */
30 #define CRUISE (0.25 * MAX_SPEED) /* velocidade de cruzeiro (~0.25 m/s) */
31
32 #define GOAL_X 1.0 /* ponto B (bate com o disco vermelho) */
33 #define GOAL_Y 1.0
34 #define GOAL_TOL 0.07 /* 7 cm: "chegou" */
35
36 /* limiares calibrados pela MEDICAO real (piso reflete -> base ~90-120): */
37 #define IR_BASE 100.0 /* leitura de base em espaco aberto */
38 #define IR_TRIG 210.0 /* >210: obstaculo de verdade -> GIRAR */
39 #define IR_CLEAR 160.0 /* <160: frente limpou (volta p/ base) */
40
41 #define K_HEADING 3.0 /* ganho P do rumo (go-to-goal) */
42 #define K_AVOID 0.05 /* ganho do desvio reativo */
43 #define SPIN 6.0 /* rad/s p/ girar no lugar no estado GIRAR */
44 #define PASS_DIST 0.30 /* m a percorrer no estado CONTORNAR */
45
46 typedef enum { NAVEGAR, GIRAR, CONTORNAR } State;
47
48 static double clampd(double v, double lo, double hi) {
49     return v < lo ? lo : (v > hi ? hi : v);
50 }
51
52 static double wrap_pi(double a) {
53     while (a > M_PI) a -= 2.0 * M_PI;
54     while (a < -M_PI) a += 2.0 * M_PI;
55     return a;
56 }
57
58 static double maxd(double a, double b) { return a > b ? a : b; }
59
60 int main(void) {
61     wb_robot_init();
62     setvbuf(stdout, NULL, _IONBF, 0); /* sem buffer: telemetria sai na hora */
63     FILE *logf = fopen("U:\\ufam\\eng_soft_rt\\controllers\\goto_avoid\\run.log", "w");
64     const int dt = (int)wb_robot_get_basic_time_step();
65
66     /* --- atuadores: rodas em modo VELOCIDADE --- */
67     WbDeviceTag left = wb_robot_get_device("left wheel motor");
68     WbDeviceTag right = wb_robot_get_device("right wheel motor");
69     wb_motor_set_position(left, INFINITY);
70     wb_motor_set_position(right, INFINITY);
71     wb_motor_set_velocity(left, 0.0);
72     wb_motor_set_velocity(right, 0.0);
73
74     /* --- sensores IR do anel --- */
75     const char *ir_names[8] = {
76         "rear left infrared sensor", "left infrared sensor",
77         "front left infrared sensor", "front infrared sensor",
78         "front right infrared sensor", "right infrared sensor",
79         "rear right infrared sensor", "rear infrared sensor"};
80     WbDeviceTag ir[8];
81     for (int i = 0; i < 8; i++) {
82         ir[i] = wb_robot_get_device(ir_names[i]);
83         wb_distance_sensor_enable(ir[i], dt);
84     }
85
86     /* --- localizacao: GPS + IMU --- */
87     WbDeviceTag gps = wb_robot_get_device("gps");
88     wb_gps_enable(gps, dt);
89     WbDeviceTag imu = wb_robot_get_device("inertial unit");
90     wb_inertial_unit_enable(imu, dt);

```

```

89
90 State state = NAVEGAR;
91 int escape_dir = 1; /* +1 = gira p/ esquerda, -1 = p/ direita */
92 double pass_x = 0, pass_y = 0;
93 const char *snames[3] = {"NAV", "GIR", "CON"};
94
95 printf("[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
96 if (logf) { fprintf(logf, "[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
97     fflush(logf); }
98 double last_print = -1.0;
99
100 /* ===== LACO DE CONTROLE ===== */
101 while (wb_robot_step(dt) != -1) {
102
103     /* ---- (1) PERCEPCAO ---- */
104     const double *pos = wb_gps_get_values(gps);
105     double yaw = wb_inertial_unit_get_roll_pitch_yaw(imu)[2];
106     double dx = GOAL_X - pos[0], dy = GOAL_Y - pos[1];
107     double dist = sqrt(dx * dx + dy * dy);
108
109     if (dist < GOAL_TOL) { /* chegou -> para tudo */
110         wb_motor_set_velocity(left, 0.0);
111         wb_motor_set_velocity(right, 0.0);
112         printf(">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1], wb_robot_get_time());
113         if (logf) { fprintf(logf, ">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1],
114             wb_robot_get_time()); fflush(logf); fclose(logf); logf = NULL; }
115         break;
116     }
117
118     double f = wb_distance_sensor_get_value(ir[3]); /* frente */
119     double fl = wb_distance_sensor_get_value(ir[2]); /* frente-esq */
120     double fr = wb_distance_sensor_get_value(ir[4]); /* frente-dir */
121     double l = wb_distance_sensor_get_value(ir[1]); /* esquerda */
122     double r = wb_distance_sensor_get_value(ir[5]); /* direita */
123     double block_l = clampd((fl - IR_BASE) + 0.5 * (l - IR_BASE), 0, 1e9);
124     double block_r = clampd((fr - IR_BASE) + 0.5 * (r - IR_BASE), 0, 1e9);
125     double front = maxd(f, maxd(fl, fr)); /* "tem parede a frente?" */
126
127     double goal_dir = atan2(dy, dx);
128     double err = wrap_pi(goal_dir - yaw);
129
130     /* ---- (2) MAQUINA DE ESTADOS ---- */
131     double forward = 0.0, steer = 0.0;
132     switch (state) {
133     case NAVEGAR:
134         /* vai pro alvo (P de rumo) + desvio reativo leve */
135         steer = K_HEADING * err + K_AVOID * (block_r - block_l);
136         forward = CRUISE;
137         forward *= (1.0 - 0.7 * clampd((f - IR_BASE) / (IR_TRIG - IR_BASE), 0, 1));
138         forward *= clampd(1.0 - 0.5 * fabs(err), 0.3, 1.0);
139         if (front > IR_TRIG) { /* parede! decide UMA vez e trava */
140             escape_dir = (block_l <= block_r) ? +1 : -1; /* gira p/ o lado livre */
141             state = GIRAR;
142         }
143         break;
144     case GIRAR:
145         /* gira no lugar, MESMA direcao, ate a frente limpar de verdade */
146         forward = 0.0;
147         steer = escape_dir * SPIN;
148         if (f < IR_CLEAR && fl < IR_CLEAR && fr < IR_CLEAR) {
149             pass_x = pos[0];

```

```

150     pass_y = pos[1];
151     state = CONTORNAR;
152 }
153 break;
154
155 case CONTORNAR:
156     /* anda reto p/ passar a caixa; so desvio leve; re-gira se topar outra */
157     forward = 0.7 * CRUISE;
158     steer = K_AVOID * (block_r - block_l);
159     if (front > IR_TRIG) {
160         escape_dir = (block_l <= block_r) ? +1 : -1;
161         state = GIRAR;
162     } else {
163         double tx = pos[0] - pass_x, ty = pos[1] - pass_y;
164         if (sqrt(tx * tx + ty * ty) > PASS_DIST)
165             state = NAVEGAR; /* contornou -> re-mira no alvo */
166     }
167     break;
168 }
169
170 /* ---- (3) CINEMATICA DIFERENCIAL ---- */
171 double vl = clampd(forward - steer, -MAX_SPEED, MAX_SPEED);
172 double vr = clampd(forward + steer, -MAX_SPEED, MAX_SPEED);
173 wb_motor_set_velocity(left, vl);
174 wb_motor_set_velocity(right, vr);
175
176 /* ---- telemetria (1x/s) ---- */
177 double t = wb_robot_get_time();
178 if (t - last_print >= 1.0) {
179     last_print = t;
180     printf("[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]=%3.0f/%3.0f/%3.0f\n",
181           rodas=("%.1f/%.1f\n",
182             snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr);
183     if (logf) { fprintf(logf, "[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]
184                   =%3.0f/%3.0f/%3.0f rodas=("%.1f/%.1f\n",
185                     snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr); fflush(logf); }
186 }
187 }
188
189 wb_robot_cleanup();
190 return 0;
191 }

```

B Núcleo de controle do robô físico (Bug2 + ponto-P)

Parâmetros compartilhados (controller_config.h) e a máquina de estados GOAL/WALL-FOLLOW/BEIRADA (controller_core.h), reutilizada tanto pelo robô físico quanto pelo simulador OpenGL.

```

1  /*
2  * controller_config.h -- parametros compartilhados pelo robo e simulador.
3  *
4  * A interface web pode editar estes #defines no robo antes de recompilar.
5  * O visualizador OpenGL tambem inclui este arquivo pelo controller_core.h.
6  */
7  #ifndef KHEPERA_CONTROLLER_CONFIG_H
8  #define KHEPERA_CONTROLLER_CONFIG_H
9
10 /* ----- chegada ----- */
11 #define ARRIVE_MM 90.0 /* raio de chegada em B */
12

```

```

13  /* ----- rumo (go-to-goal) ----- */
14  #define KP_HEAD 1.2 /* ganho de rumo (erro rad -> diff) */
15  #define HEAD_SLOW 1.2 /* rad de erro p/ (quase) girar parado */
16
17  /* ----- rastreamento da linha: ponto de controle virtual P ----- */
18  #define USE_POINT_P 1 /* 1 = ponto-P (segue a linha); 0 = mira em B */
19  #define LOOKAHEAD_MM 120.0 /* "d": distancia do ponto P */
20  #define K_CT 1.0 /* ganho proporcional do erro lateral */
21  #define KI_CT 0.0015 /* ganho integral do erro lateral */
22  #define CT_INT_MAX 250.0 /* limite anti-windup do integrador */
23
24  /* ----- IR (em "excesso" = leitura - IR_BASE) ----- */
25  #define IR_BASE 120
26  #define FRONT_SPAN 450.0
27  #define FRONT_BLOCK 200 /* frente travada -> entra em WALLF */
28  #define FRONT_CRIT 320 /* quina interna -> pivota p/ longe */
29  #define FRONT_CLEAR 90 /* frente livre p/ poder largar a parede */
30  #define K_AVOID 0.0028 /* desvio lateral ainda no estado GOAL */
31
32  /* ----- seguimento de parede ----- */
33  #define WALL_TARGET 150 /* distancia-alvo do lado seguido (excesso) */
34  #define WALL_LOST 45 /* abaixo disso considera quina externa */
35  #define WALL_MIN_MM 180.0 /* minimo de contorno antes de relargar linha */
36  #define WALL_NO_PROGRESS_MM 700.0 /* inverte contorno se ficar orbitando */
37  #define KP_WALL 0.0045
38  #define STEER_HARD 0.75
39
40  /* ----- leave condition (Bug2) ----- */
41  #define LINE_EPS 60.0 /* mm: "esta sobre a linha A->B" */
42  #define PROGRESS_MM 40.0 /* precisa estar mais perto de B que no hit */
43
44  /* ----- seguranca / filtro ----- */
45  #define TURN_SPEED 0.5
46  #define DIFF_MAX 0.90
47  #define ALPHA 0.4
48  #define CLIFF_THR 500 /* IR de chao abaixo disso = beirada */
49
50  #endif

```

```

1  /*
2   * controller_core.h -- nucleo compartilhado do controlador Bug2 do Khepera.
3   *
4   * Este arquivo nao depende do hardware. Ele recebe pose, sensores e tempo,
5   * devolve comando diferencial normalizado e telemetria logica. O main.c do
6   * robo real e o visualizador/simulador OpenGL incluem este mesmo header, para
7   * testar localmente exatamente a logica que sera compilada junto do robo.
8   */
9  #ifndef KHEPERA_CONTROLLER_CORE_H
10 #define KHEPERA_CONTROLLER_CORE_H
11
12 #include <math.h>
13 #include <string.h>
14
15 #include "controller_config.h"
16
17 #ifndef M_PI
18 #define M_PI 3.14159265358979323846
19 #endif
20
21 #ifndef ARRIVE_MM
22 #define ARRIVE_MM 90.0
23 #endif
24 #ifndef KP_HEAD

```

```
25 #define KP_HEAD 1.2
26 #endif
27 #ifndef HEAD_SLOW
28 #define HEAD_SLOW 1.2
29 #endif
30 #ifndef LOOKAHEAD_MM
31 #define LOOKAHEAD_MM 120.0
32 #endif
33 #ifndef USE_POINT_P
34 #define USE_POINT_P 1
35 #endif
36 #ifndef K_CT
37 #define K_CT 1.0
38 #endif
39 #ifndef KI_CT
40 #define KI_CT 0.0015
41 #endif
42 #ifndef CT_INT_MAX
43 #define CT_INT_MAX 250.0
44 #endif
45 #ifndef IR_BASE
46 #define IR_BASE 120
47 #endif
48 #ifndef FRONT_SPAN
49 #define FRONT_SPAN 450.0
50 #endif
51 #ifndef FRONT_BLOCK
52 #define FRONT_BLOCK 200
53 #endif
54 #ifndef FRONT_CRIT
55 #define FRONT_CRIT 320
56 #endif
57 #ifndef FRONT_CLEAR
58 #define FRONT_CLEAR 90
59 #endif
60 #ifndef K_AVOID
61 #define K_AVOID 0.0028
62 #endif
63 #ifndef K_SIDE_AVOID
64 #define K_SIDE_AVOID 0.0065
65 #endif
66 #ifndef FRONT_TURN_BIAS
67 #define FRONT_TURN_BIAS 0.42
68 #endif
69 #ifndef SIDE_CRIT
70 #define SIDE_CRIT 260
71 #endif
72 #ifndef WALL_TARGET
73 #define WALL_TARGET 150
74 #endif
75 #ifndef WALL_LOST
76 #define WALL_LOST 45
77 #endif
78 #ifndef WALL_MIN_MM
79 #define WALL_MIN_MM 180.0
80 #endif
81 #ifndef WALL_NO_PROGRESS_MM
82 #define WALL_NO_PROGRESS_MM 1400.0
83 #endif
84 #ifndef KP_WALL
85 #define KP_WALL 0.0045
86 #endif
87 #ifndef STEER_HARD
```

```

88 #define STEER_HARD 0.75
89 #endif
90 #ifndef LINE_EPS
91 #define LINE_EPS 60.0
92 #endif
93 #ifndef PROGRESS_MM
94 #define PROGRESS_MM 40.0
95 #endif
96 #ifndef TURN_SPEED
97 #define TURN_SPEED 0.5
98 #endif
99 #ifndef DIFF_MAX
100 #define DIFF_MAX 0.90
101 #endif
102 #ifndef ALPHA
103 #define ALPHA 0.4
104 #endif
105 #ifndef CLIFF_THR
106 #define CLIFF_THR 500
107 #endif
108
109 typedef enum {
110     KHEP_GOAL = 0,
111     KHEP_WALLF = 1,
112     KHEP_BEIRADA = 2
113 } KhepState;
114
115 typedef struct {
116     double ax, ay, bx, by;
117     KhepState state;
118     double fwd_s, diff_s;
119     double cliff_start;
120     int wall_left;
121     double hit_dgoal, wallf_dist;
122     double best_wall_dgoal, no_progress_dist;
123     int left_line;
124     double ct_int;
125 } KhepController;
126
127 typedef struct {
128     double forward, diff;
129     double fwd_s, diff_s;
130     double d_goal, d_line, herr;
131     double ux, uy;
132     int F, FL, FR, L, R;
133     int fb, Lc, Rc;
134     int wall_left;
135     int entered_wall;
136     int left_wall;
137     int leave_by_goal;
138     int beirada;
139     KhepState state;
140 } KhepOutput;
141
142 static double khep_clampd(double v, double lo, double hi) {
143     return v < lo ? lo : (v > hi ? hi : v);
144 }
145
146 static double khep_wrap(double a) {
147     while (a > M_PI) a -= 2.0 * M_PI;
148     while (a < -M_PI) a += 2.0 * M_PI;
149     return a;
150 }

```

```

151
152 static int khep_excess(int v) {
153     int e = v - IR_BASE;
154     return e > 0 ? e : 0;
155 }
156
157 static void khep_ctrl_set_segment(KhepController *c,
158     double ax, double ay,
159     double bx, double by) {
160     c->ax = ax;
161     c->ay = ay;
162     c->bx = bx;
163     c->by = by;
164     c->state = KHEP_GOAL;
165     c->fwd_s = 0.0;
166     c->diff_s = 0.0;
167     c->cliff_start = 0.0;
168     c->wall_left = 0;
169     c->hit_dgoal = 0.0;
170     c->wallf_dist = 0.0;
171     c->best_wall_dgoal = 0.0;
172     c->no_progress_dist = 0.0;
173     c->left_line = 0;
174     c->ct_int = 0.0;
175 }
176
177 static void khep_ctrl_init(KhepController *c, double bx, double by) {
178     memset(c, 0, sizeof(*c));
179     khep_ctrl_set_segment(c, 0.0, 0.0, bx, by);
180 }
181
182 static KhepOutput khep_ctrl_step(KhepController *c,
183     double x, double y, double th,
184     double dc, double dt, double t_now,
185     int rawF, int rawFL, int rawFR,
186     int rawL, int rawR, int gmin) {
187     KhepOutput out;
188     double vx, vy, vlen, px, py, bearing;
189     double forward = 0.0, diff = 0.0;
190
191     memset(&out, 0, sizeof(out));
192
193     out.F = khep_excess(rawF);
194     out.FL = khep_excess(rawFL);
195     out.FR = khep_excess(rawFR);
196     out.L = khep_excess(rawL);
197     out.R = khep_excess(rawR);
198     out.Lc = out.FL > out.L ? out.FL : out.L;
199     out.Rc = out.FR > out.R ? out.FR : out.R;
200     out.fb = out.F;
201     if (out.FL > out.fb) out.fb = out.FL;
202     if (out.FR > out.fb) out.fb = out.FR;
203
204     vx = c->bx - c->ax;
205     vy = c->by - c->ay;
206     vlen = hypot(vx, vy);
207     if (vlen < 1e-6) vlen = 1e-6;
208     out.ux = vx / vlen;
209     out.uy = vy / vlen;
210     px = x - c->ax;
211     py = y - c->ay;
212     out.d_line = out.ux * py - out.uy * px;
213     out.d_goal = hypot(c->bx - x, c->by - y);

```

```

214 bearing = atan2(c->by - y, c->bx - x);
215 out.herr = khep_wrap(bearing - th);
216
217 if (c->state != KHEP_BEIRADA && gmin < CLIFF_THR) {
218     c->state = KHEP_BEIRADA;
219     c->cliff_start = t_now;
220 }
221
222 if (c->state == KHEP_BEIRADA) {
223     double e = t_now - c->cliff_start;
224     out.beirada = 1;
225     if (e < 0.5) {
226         forward = -0.6;
227         diff = 0.0;
228     } else if (e < 1.3) {
229         forward = 0.0;
230         diff = TURN_SPEED;
231     } else {
232         c->state = KHEP_GOAL;
233         c->fwd_s = 0.0;
234         c->diff_s = 0.0;
235         c->ct_int = 0.0;
236         c->wallf_dist = 0.0;
237     }
238     out.forward = forward;
239     out.diff = diff;
240     out.fwd_s = forward;
241     out.diff_s = diff;
242     out.state = c->state;
243     out.wall_left = c->wall_left;
244     return out;
245 }
246
247 if (c->state == KHEP_GOAL) {
248     double theta_line = atan2(out.uy, out.ux);
249     double ct_cmd = K_CT * out.d_line + KI_CT * c->ct_int;
250     double cross_corr = atan2(ct_cmd, LOOKAHEAD_MM);
251     double head_ref = khep_wrap(theta_line - cross_corr);
252     double herr_ctrl = USE_POINT_P ? khep_wrap(head_ref - th) : out.herr;
253     double left_pressure = (double)out.FL + 0.75 * (double)out.L;
254     double right_pressure = (double)out.FR + 0.75 * (double)out.R;
255     double side_guard = 0.0;
256     if (out.d_goal < 650.0) {
257         herr_ctrl = out.herr;
258     }
259     double fr_scale;
260
261     if (out.F > FRONT_CLEAR || out.Lc > SIDE_CRIT || out.Rc > SIDE_CRIT) {
262         side_guard += K_AVOID * (double)(out.Lc - out.Rc);
263     }
264     if (out.Lc > SIDE_CRIT || out.Rc > SIDE_CRIT) {
265         side_guard += K_SIDE_AVOID * (left_pressure - right_pressure);
266     }
267     if (out.F > FRONT_CLEAR) {
268         if (right_pressure > left_pressure + 25.0) {
269             side_guard -= FRONT_TURN_BIAS;
270         } else if (left_pressure > right_pressure + 25.0) {
271             side_guard += FRONT_TURN_BIAS;
272         } else {
273             side_guard += (out.herr >= 0.0) ? -FRONT_TURN_BIAS : FRONT_TURN_BIAS;
274         }
275     }
276 }

```



```

277     diff = -KP_HEAD * herr_ctrl + side_guard;
278     diff = kkep_clampd(diff, -DIFF_MAX, DIFF_MAX);
279     forward = kkep_clampd(1.0 - fabs(herr_ctrl) / HEAD_SLOW, 0.12, 1.0);
280     fr_scale = kkep_clampd(1.0 - out.fb / FRONT_SPAN, 0.25, 1.0);
281     if (fr_scale < forward) forward = fr_scale;
282     if (out.Lc > SIDE_CRIT || out.Rc > SIDE_CRIT) {
283         double side_scale = kkep_clampd(1.0 - ((double)(out.Lc > out.Rc ? out.Lc : out.Rc)) /
284             FRONT_SPAN,
285             0.10, 0.70);
286         if (side_scale < forward) forward = side_scale;
287     }
288     if (fabs(diff) < DIFF_MAX) {
289         c->ct_int = kkep_clampd(c->ct_int + out.d_line * dt,
290             -CT_INT_MAX, CT_INT_MAX);
291     }
292
293     if ((out.F > FRONT_BLOCK || (out.FL > FRONT_CRIT && out.FR > FRONT_CRIT)) &&
294         (out.d_goal > 3.0 * ARRIVE_MM || out.fb > FRONT_CRIT)) {
295         c->state = KHEP_WALLF;
296         c->hit_dgoal = out.d_goal;
297         c->best_wall_dgoal = out.d_goal;
298         c->no_progress_dist = 0.0;
299         c->wallf_dist = 0.0;
300         c->left_line = 0;
301         c->ct_int = 0.0;
302         if (out.Lc == out.Rc)
303             c->wall_left = (out.herr < 0.0);
304         else
305             c->wall_left = (out.Lc > out.Rc);
306         out.entered_wall = 1;
307     }
308     } else {
309         int side, other, err;
310         double goal_diff;
311         c->wallf_dist += fabs(dc);
312         if (out.d_goal < c->best_wall_dgoal - 25.0) {
313             c->best_wall_dgoal = out.d_goal;
314             c->no_progress_dist = 0.0;
315         } else {
316             c->no_progress_dist += fabs(dc);
317         }
318         if (c->no_progress_dist > WALL_NO_PROGRESS_MM) {
319             c->wall_left = !c->wall_left;
320             c->hit_dgoal = out.d_goal;
321             c->best_wall_dgoal = out.d_goal;
322             c->wallf_dist = 0.0;
323             c->no_progress_dist = 0.0;
324             c->left_line = 0;
325         }
326
327         side = c->wall_left ? out.Lc : out.Rc;
328         other = c->wall_left ? out.Rc : out.Lc;
329         err = WALL_TARGET - side;
330
331         if (c->wallf_dist > WALL_MIN_MM &&
332             out.F < FRONT_CLEAR && out.FL < FRONT_BLOCK && out.FR < FRONT_BLOCK &&
333             (fabs(out.herr) < 1.75 || c->no_progress_dist > 0.45 * WALL_NO_PROGRESS_MM) &&
334             out.d_goal < c->hit_dgoal + 320.0) {
335             c->state = KHEP_GOAL;
336             c->fwd_s = 0.0;
337             c->diff_s = 0.0;
338             c->ct_int = 0.0;

```

```

339     out.left_wall = 1;
340     out.leave_by_goal = 1;
341 }
342
343 if (out.F > FRONT_CRIT && out.Lc > SIDE_CRIT && out.Rc > SIDE_CRIT) {
344     forward = -0.10;
345     diff = (out.Lc > out.Rc) ? +STEER_HARD : -STEER_HARD;
346 } else if (other > SIDE_CRIT) {
347     forward = 0.08;
348     diff = c->wall_left ? -STEER_HARD : +STEER_HARD;
349 } else if (side > FRONT_CRIT) {
350     forward = 0.12;
351     diff = c->wall_left ? +STEER_HARD : -STEER_HARD;
352 } else if (side < WALL_LOST && out.F < FRONT_CLEAR) {
353     forward = 0.22;
354     diff = c->wall_left ? -0.55 * STEER_HARD : +0.55 * STEER_HARD;
355 } else if (out.F > FRONT_CRIT) {
356     forward = 0.06;
357     diff = c->wall_left ? +STEER_HARD : -STEER_HARD;
358 } else {
359     double d = KP_WALL * (double)err;
360     forward = khep_clampd(1.0 - out.F / FRONT_SPAN, 0.30, 0.85);
361     diff = c->wall_left ? -d : +d;
362 }
363 goal_diff = khep_clampd(-0.65 * KP_HEAD * out.herr, -DIFF_MAX, DIFF_MAX);
364 if (c->wallf_dist > 2.0 * WALL_MIN_MM && out.F < FRONT_CLEAR && fabs(out.herr) < 1.25) {
365     diff = 0.70 * diff + 0.30 * goal_diff;
366 }
367
368 if (fabs(out.d_line) > 1.5 * LINE_EPS) c->left_line = 1;
369 {
370     int on_leave_line = (c->left_line || c->wallf_dist > WALL_MIN_MM) &&
371         fabs(out.d_line) < LINE_EPS;
372     int target_clear = c->wallf_dist > 2.0 * WALL_MIN_MM &&
373         fabs(out.herr) < 0.90 &&
374         out.fb < FRONT_CLEAR;
375     int near_goal_release = out.d_goal < 3.0 * ARRIVE_MM &&
376         out.fb < FRONT_BLOCK;
377     if (near_goal_release ||
378         ((on_leave_line || target_clear) &&
379         out.d_goal < c->hit_dgoal - PROGRESS_MM &&
380         out.fb < FRONT_CLEAR)) {
381         c->state = KHEP_GOAL;
382         c->fwd_s = 0.0;
383         c->diff_s = 0.0;
384         out.left_wall = 1;
385         out.leave_by_goal = (target_clear || near_goal_release) && !on_leave_line;
386     }
387 }
388 }
389
390 diff = khep_clampd(diff, -DIFF_MAX, DIFF_MAX);
391 c->fwd_s = (1.0 - ALPHA) * c->fwd_s + ALPHA * forward;
392 c->diff_s = (1.0 - ALPHA) * c->diff_s + ALPHA * diff;
393
394 out.forward = forward;
395 out.diff = diff;
396 out.fwd_s = c->fwd_s;
397 out.diff_s = c->diff_s;
398 out.state = c->state;
399 out.wall_left = c->wall_left;
400 return out;
401 }

```

```
#endif
```

C Programa principal do robô físico

Laço embarcado com odometria, modos de operação (missão, *loop*, *teleop*, diagnóstico) e integração com a interface web (`khepera_real/patrolha/main.c`).

```
1  /*
2  * main.c -- Khepera IV: IR DO PONTO A AO PONTO B desviando de obstaculos
3  * =====
4  * OBJETIVO: navegar de A (origem, onde o robo liga) ate B (alvo em mm),
5  * contornando qualquer obstaculo e VOLTANDO para a linha A->B -- ou seja,
6  * um go-to-goal de verdade, nao um passeio reativo.
7  *
8  * ALGORITMO: Bug2 (Lumelsky & Stepanov, 1987) -- o padrao de competicao p/
9  * "chegar num ponto com sensores locais":
10 * 1) MIRA em B e anda reto pela LINHA A->B (controle de rumo por odometria).
11 * 2) Bateu num obstaculo? -> SEGUE A PAREDE (wall-following) contornando-o,
12 * MAS lembrando a linha A->B.
13 * 3) LEAVE CONDITION: no instante em que RECRUZA a linha A->B num ponto MAIS
14 * PERTO de B do que o ponto onde bateu -> larga a parede e volta a mirar B.
15 * 4) Repete ate chegar em B.
16 * O passo 3 e o que faz o robo VOLTAR PRO EIXO em vez de sair vagando (era o
17 * que faltava: sem alvo, o reativo puro nunca "volta pro meio").
18 *
19 * POSE (x,y,theta) por ODOMETRIA (encoders): 147,4 pulsos/mm, base 105,4 mm.
20 * (Sem GPS como no Webots -> dead reckoning pelos encoders das rodas.)
21 *
22 * Convencao de giro (do driver: L=fwd+diff, R=fwd-diff):
23 * diff > 0 => vira DIREITA (theta DIMINUI) diff < 0 => ESQUERDA (theta sobe)
24 *
25 * Uso:
26 * ./patrolha vai ate (700,0) mm e para
27 * ./patrolha --gx 1000 --gy 300 alvo B em (1000,300) mm
28 * ./patrolha --loop vai-e-volta A->B pra sempre (bom pra demo)
29 * ./patrolha --diag so le sensores/encoders, nao move
30 * ./patrolha --teleop controle manual pela serial (comandos v/s/q)
31 * ./patrolha --mission recebe waypoints "goto X Y" pela serial, com a
32 * odometria continua (usado pelo planejador A* da
33 * interface sobre o mapa gerado)
34 * ./patrolha --slow metade da velocidade linear -> contorno lento e
35 * limpo (usado na geracao automatica de mapa A->B)
36 * ./patrolha -t 0.03 trim se o robo puxa pra um lado
37 */
38 #include "khepera4.h"
39 #include "commandline.h"
40 #include <stdio.h>
41 #include <stdlib.h>
42 #include <string.h>
43 #include <unistd.h>
44 #include <signal.h>
45 #include <math.h>
46 #include <sys/time.h>
47 #include <sys/select.h>
48
49 /* ----- odometria ----- */
50 #define PULSES_PER_MM 147.4
51 #define WHEELBASE_MM 105.4
52
53 /* ----- alvo / chegada ----- */
54 #define GOAL_X_DEF 700.0 /* B padrao (mm a frente de A) */
```

```

55 #define GOAL_Y_DEF 0.0
56
57 /* ----- geral ----- */
58 #define SPEED 12000
59 #define WAIT_US 40000
60 #define BATT_LOW_MV 6300
61
62 #include "controller_config.h"
63
64 static volatile sig_atomic_t running = 1;
65 static void on_sigint(int s){ (void)s; running = 0; }
66 static double clampd(double v,double lo,double hi){ return v<lo?lo:(v>hi?hi:v); }
67 static int excess(int v){ int e=v-IR_BASE; return e>0?e:0; }
68 static double now_s(void){ struct timeval t; gettimeofday(&t,NULL); return t.tv_sec + t.tv_usec
    /1e6; }
69 static double wrap(double a){ while(a>M_PI)a-=2*M_PI; while(a<-M_PI)a+=2*M_PI; return a; }
70 #define IRP(i) (khepera4.infrared_proximity.sensor[i])
71
72 #include "controller_core.h"
73
74 static void leds(int r,int g,int b){
75     khepera4_dspic_write_register8(I2C_RGB_FL_R,r); khepera4_dspic_write_register8(I2C_RGB_FL_G,g)
76     ; khepera4_dspic_write_register8(I2C_RGB_FL_B,b);
77     khepera4_dspic_write_register8(I2C_RGB_FR_R,r); khepera4_dspic_write_register8(I2C_RGB_FR_G,g)
78     ; khepera4_dspic_write_register8(I2C_RGB_FR_B,b);
79     khepera4_dspic_write_register8(I2C_RGB_B_R,r); khepera4_dspic_write_register8(I2C_RGB_B_G,g);
80     khepera4_dspic_write_register8(I2C_RGB_B_B,b);
81 }
82
83 static void diag(void){
84     printf("DIAG: sensores + encoders (sem mover). Ctrl-C p/ sair.\n");
85     while(running){
86         khepera4_infrared_proximity(); khepera4_battery(); khepera4_drive_get_current_position();
87         printf("IR[BL L FL F FR R BR B]=%d %d %d %d %d %d %d %d | chao[%d %d %d %d] | enc L/R=%d
            %d | bat=%umV\n",
88             IRP(0),IRP(1),IRP(2),IRP(3),IRP(4),IRP(5),IRP(6),IRP(7), IRP(8),IRP(9),IRP(10),IRP
            (11),
89             khepera4.motor_left.current_position, khepera4.motor_right.current_position,
90             khepera4.battery.voltage);
91         fflush(stdout); usleep(400000);
92     }
93 }
94
95 static int teleop_read_command(double *fwd, double *diff, int *quit){
96     fd_set rfd;
97     struct timeval tv;
98     char line[96], op[16];
99     double a=0, b=0;
100
101     FD_ZERO(&rfd);
102     FD_SET(STDIN_FILENO, &rfd);
103     tv.tv_sec = 0;
104     tv.tv_usec = 0;
105     if(select(STDIN_FILENO + 1, &rfd, NULL, NULL, &tv) <= 0) return 0;
106     if(!fgets(line, sizeof(line), stdin)) return 0;
107     if(sscanf(line, " %15s %lf %lf", op, &a, &b) < 1) return 0;
108
109     if(strcmp(op, "v") == 0 && sscanf(line, " %*s %lf %lf", &a, &b) == 2){
110         *fwd = clampd(a, -1.0, 1.0);
111         *diff = clampd(b, -DIFF_MAX, DIFF_MAX);
112         return 1;
113     }
114     if(strcmp(op, "s") == 0 || strcmp(op, "stop") == 0){

```

```

111     *fwd = 0.0;
112     *diff = 0.0;
113     return 1;
114 }
115 if(strcmp(op, "q") == 0 || strcmp(op, "quit") == 0){
116     *fwd = 0.0;
117     *diff = 0.0;
118     *quit = 1;
119     return 1;
120 }
121 return 0;
122 }
123
124 /* Modo missao: le do stdin (nao bloqueia) uma das linhas:
125  * goto X Y -> novo alvo em (X,Y) mm (retorna 1, preenche *gx,*gy)
126  * stop -> para e aguarda proximo goto (retorna 2)
127  * quit / q -> encerra a missao (retorna 3)
128  * Retorna 0 quando nao ha comando. Usado pela interface p/ enviar os
129  * waypoints do planejador A* sobre o mapa, com a odometria continua. */
130 static int mission_read(double *gx, double *gy){
131     fd_set rfds; struct timeval tv; char line[96], op[16]; double a=0,b=0;
132     FD_ZERO(&rfds); FD_SET(STDIN_FILENO,&rfds); tv.tv_sec=0; tv.tv_usec=0;
133     if(select(STDIN_FILENO+1,&rfds,NULL,NULL,&tv)<=0) return 0;
134     if(!fgets(line,sizeof(line),stdin)) return 0;
135     if(sscanf(line, "%15s",op)<1) return 0;
136     if(strcmp(op,"goto")==0 && sscanf(line, "%s %lf %lf",&a,&b)==2){ *gx=a; *gy=b; return 1; }
137     if(strcmp(op,"stop")==0) return 2;
138     if(strcmp(op,"quit")==0 || strcmp(op,"q")==0) return 3;
139     return 0;
140 }
141
142 static void teleop(void){
143     double target_fwd = 0.0, target_diff = 0.0;
144     double last_cmd = now_s();
145     long tick = 0;
146     int batt_tick = 0;
147
148     khepera4_drive_start();
149     khepera4_drive_reset_position(); usleep(120000);
150     khepera4_drive_get_current_position();
151     int enc_l0 = khepera4.motor_left.current_position;
152     int enc_r0 = khepera4.motor_right.current_position;
153     double x=0, y=0, th=0;
154     leds(35, 25, 0);
155     printf("TELEOP: comandos pela serial: v <fwd -1..1> <diff -0.9..0.9>, s=para, q=sair.\n");
156     printf("TELEOP: seguranca ativa: sem comando por 0.8s => motores parados.\n");
157     fflush(stdout);
158
159     while(running){
160         int quit = 0;
161         if(teleop_read_command(&target_fwd, &target_diff, &quit)){
162             last_cmd = now_s();
163             if(quit) break;
164         }
165
166         khepera4_infrared_proximity();
167         khepera4_drive_get_current_position();
168         if((batt_tick++ % 25)==0) khepera4_battery();
169
170         int el = khepera4.motor_left.current_position;
171         int er = khepera4.motor_right.current_position;
172         double dl = (double)(el-enc_l0)/PULSES_PER_MM;
173         double dr = (double)(er-enc_r0)/PULSES_PER_MM;

```

```

174     enc_l0=el; enc_r0=er;
175     double dc = 0.5*(dl+dr);
176     double dth = (dr-dl)/WHEELBASE_MM;
177     x += dc*cos(th+0.5*dth);
178     y += dc*sin(th+0.5*dth);
179     th = wrap(th+dth);
180
181     int F=excess(IRP(3)), FL=excess(IRP(2)), FR=excess(IRP(4));
182     int gmin=IRP(8); if (IRP(9)<gmin)gmin=IRP(9); if (IRP(10)<gmin)gmin=IRP(10); if (IRP(11)<
        gmin)gmin=IRP(11);
183     double fwd = target_fwd, diff = target_diff;
184     const char *guard = "OK";
185
186     if(now_s() - last_cmd > 0.8){
187         fwd = 0.0;
188         diff = 0.0;
189         guard = "STALE";
190     }
191     if(gmin < CLIFF_THR && fwd > 0.0){
192         fwd = 0.0;
193         guard = "BEIRADA";
194     }
195
196     khepera4_drive_set_speed_differential(SPEED, fwd, diff);
197
198     if(++tick % 8 == 0){
199         printf("[MANUAL] pose=(%.0f,%.0f,%.0fd) fwd=%+.2f diff=%+.2f guard=%s | F/FL/FR=%d/%d
            /%d | chao=%d bat=%umV\n",
200             x, y, th*180/M_PI, fwd, diff, guard, F+IR_BASE, FL+IR_BASE, FR+IR_BASE, gmin,
            khepera4.battery.voltage);
201         fflush(stdout);
202     }
203     usleep(WAIT_US);
204 }
205
206 khepera4_drive_set_speed_differential(SPEED, 0.0, 0.0);
207 khepera4_drive_stop();
208 khepera4_drive_idle();
209 leds(0,0,0);
210 printf("\nTELEOP encerrado. Motores parados.\n");
211 fflush(stdout);
212 }
213
214 int main(int argc, char *argv[]){
215     cmdline_init(); cmdline_parse(argc, argv);
216     signal(SIGINT,on_sigint); signal(SIGTERM,on_sigint); signal(SIGHUP,on_sigint);
217
218     khepera4_init();
219     if (commandline_option_provided("-d","--diag")){ diag(); return 0; }
220     if (commandline_option_provided("-m","--teleop")){ teleop(); return 0; }
221
222     double trim = cmdline_option_value_float("-t","--trim",0.0);
223     double gx = cmdline_option_value_float("-x","--gx",GOAL_X_DEF);
224     double gy = cmdline_option_value_float("-y","--gy",GOAL_Y_DEF);
225     int loop = cmdline_option_provided("-l","--loop");
226     int mission = cmdline_option_provided("-M","--mission"); /* waypoints via stdin */
227     int slow = cmdline_option_provided("-w","--slow"); /* contorno lento p/ mapear */
228     double vscale = slow ? 0.5 : 1.0; /* escala da velocidade linear */
229
230     khepera4_drive_start();
231     khepera4_drive_reset_position(); usleep(120000);
232     khepera4_drive_get_current_position();
233     int enc_l0 = khepera4.motor_left.current_position;

```

```

234     int enc_r0 = khepera4.motor_right.current_position;
235
236     KhepController ctrl;
237     double x=0, y=0, th=0; /* pose por odometria (mm, mm, rad) */
238
239     khep_ctrl_init(&ctrl, gx, gy);
240     int led_state=-1;
241     int have_goal = !mission; /* modo missao comeca sem alvo (aguarda goto) */
242     long tick=0; int batt_tick=0;
243     double t_prev = now_s();
244
245     leds(0,0,30);
246     if(mission)
247         printf("MISSAO: aguardando 'goto X Y' pela serial (stop/quit). Ctrl-C p/ parar.\n");
248     else
249         printf("A->B (Bug2, odometria). B=(%.0f,%.0f)mm loop=%d trim=%.3f. Ctrl-C p/ parar.\n",
                ctrl.bx,ctrl.by,loop,trim);
250
251     while(running){
252         double t_now=now_s(); double dt=t_now-t_prev; t_prev=t_now;
253         if(dt<=0||dt>0.5) dt=WAIT_US/1e6;
254
255         /* ----- leitura de sensores + odometria ----- */
256         khepera4_infrared_proximity();
257         khepera4_drive_get_current_position();
258         if((batt_tick++ % 50)==0) khepera4_battery();
259
260         int el = khepera4.motor_left.current_position;
261         int er = khepera4.motor_right.current_position;
262         double dl = (double)(el-enc_l0)/PULSES_PER_MM; /* mm roda esq */
263         double dr = (double)(er-enc_r0)/PULSES_PER_MM; /* mm roda dir */
264         enc_l0=el; enc_r0=er;
265         double dc = 0.5*(dl+dr);
266         double dth = (dr-dl)/WHEELBASE_MM; /* dir>esq => theta sobe (esq) */
267         x += dc*cos(th+0.5*dth);
268         y += dc*sin(th+0.5*dth);
269         th = wrap(th+dth);
270
271         /* ----- modo missao: recebe waypoints do planejador (A* no mapa) ----- */
272         if(mission){
273             double ngx,ngy; int mc=mission_read(&ngx,&ngy);
274             if(mc==1){ /* goto X Y: novo leg A(atual)->B */
275                 khep_ctrl_set_segment(&ctrl, x, y, ngx, ngy);
276                 have_goal=1;
277                 printf(">> GOTO (%.0f,%.0f)\n", ctrl.bx,ctrl.by); fflush(stdout);
278             } else if(mc==2){ /* stop: aguarda proximo goto */
279                 have_goal=0; printf(">> STOP\n"); fflush(stdout);
280             } else if(mc==3){ /* quit: encerra a missao */
281                 break;
282             }
283             if(!have_goal){
284                 khepera4_drive_set_speed_differential(SPEED,0,0);
285                 usleep(WAIT_US); continue;
286             }
287         }
288
289         int rawF=IRP(3), rawFL=IRP(2), rawFR=IRP(4), rawL=IRP(1), rawR=IRP(5);
290         int gmin=IRP(8); if(IRP(9)<gmin)gmin=IRP(9); if(IRP(10)<gmin)gmin=IRP(10); if(IRP(11)<
            gmin)gmin=IRP(11);
291         KhepOutput co = khep_ctrl_step(&ctrl, x, y, th, dc, dt, t_now,
292                                         rawF, rawFL, rawFR, rawL, rawR, gmin);
293
294         if(co.entered_wall){

```

```

295     printf(">> WALLF: parede a %s (hit d_goal=%.0f, herr=%.0fdeg)\n",
296           co.wall_left?"ESQUERDA":"DIREITA", co.d_goal, co.herr*180/M_PI); fflush(stdout);
297 }
298 if(co.left_wall){
299     if(co.leave_by_goal)
300         printf("<< alvo livre e progresso feito (d_goal=%.0f) -> GOAL\n", co.d_goal);
301     else
302         printf("<< recruzou a linha mais perto de B (d_goal=%.0f) -> GOAL\n", co.d_goal);
303     fflush(stdout);
304 }
305
306 /* ----- CHEGOU? ----- */
307 if(co.d_goal < ARRIVE_MM){
308     khepera4_drive_set_speed_differential(SPEED,0,0);
309     printf("== CHEGOU em (%.0f,%.0f) pose=(%.0f,%.0f,%.0fdeg) ==\n", ctrl.bx,ctrl.by,x,y,
310           th*180/M_PI); fflush(stdout);
311     if(mission){ /* chegou no waypoint: aguarda o proximo goto */
312         have_goal=0; khep_ctrl_set_segment(&ctrl, x, y, x, y);
313         leds(0,40,0); usleep(200000); continue;
314     }
315     if(!loop) break;
316     { /* inverte o segmento */
317         double old_ax=ctrl.ax, old_ay=ctrl.ay;
318         khep_ctrl_set_segment(&ctrl, ctrl.bx, ctrl.by, old_ax, old_ay);
319     }
320     leds(0,40,0); usleep(600000); continue;
321 }
322
323 /* ----- ANTI-QUEDA (prioridade maxima) ----- */
324 if(co.beirada){
325     if(led_state!=3){ leds(40,0,0); led_state=3; }
326     khepera4_drive_set_speed_differential(SPEED, co.fwd_s, co.diff_s);
327     if(++tick%8==0){ printf("[BEIRADA] chao=%d\n",gmin); fflush(stdout); }
328     usleep(WAIT_US); continue;
329 }
330
331 if(co.state==KHEP_GOAL){
332     if(led_state!=0){ leds(0,30,0); led_state=0; } /* verde: rumo a B */
333 } else {
334     if(led_state!=2){ leds(0,0,40); led_state=2; } /* azul: contornando */
335 }
336
337 khepera4_drive_set_speed_differential(SPEED, co.fwd_s*vscale, co.diff_s + trim);
338
339 if(++tick%15==0){
340     printf("[%s%s] pose=(%.0f,%.0f,%.0fd) d_goal=%.0f d_line=%+.0f herr=%+.0fd | F/FL/FR=%
341           d/%d/%d | chao=%d bat=%umV\n",
342           co.state==KHEP_GOAL?"GOAL ":"WALLF", co.state==KHEP_WALLF?(co.wall_left?"-E":"-D
343           "):"",
344           x,y,th*180/M_PI, co.d_goal, co.d_line, co.herr*180/M_PI,
345           co.F+IR_BASE,co.FL+IR_BASE,co.FR+IR_BASE, gmin, khepera4.battery.voltage);
346     if(khepera4.battery.voltage>0 && khepera4.battery.voltage<BATT_LOW_MV) printf(" !!
347           BATERIA BAIXA !!\n");
348     fflush(stdout);
349 }
350     usleep(WAIT_US);
351 }
352
353 khepera4_drive_stop(); khepera4_drive_idle(); leds(0,0,0);
354 printf("\nMotores parados. Encerrado com seguranca.\n");
355 return 0;
356 }

```


D Planejador global A^*

Planejamento sobre a grade de ocupação, com dilatação pelo raio do robô e simplificação por linha-de-visada (interface/planner.py).

```

1  """Planejador de caminho A* sobre o mapa de ocupacao gerado pela interface.
2
3  O mapa (`grid.json`) e esparso: so tras celulas observadas, com `state` em
4  {free, visited, occupied}. Aqui:
5  - `occupied` (dilatado pelo raio do robo) = bloqueado;
6  - todo o resto (free, visited e o NAO observado) = livre (otimista) -- o
7    Bug2 reativo do robo cuida de qualquer obstaculo nao mapeado.
8
9  Uso principal: `plan_waypoints(grid, start_xy, goal_xy)` -> lista de (x,y) em mm.
10 Funcao pura, sem dependencias externas, testavel offline.
11 """
12 import heapq
13 import math
14
15 ROBOT_RADIUS_MM = 90.0 # ~meia base (105 mm) + folga; dilata os obstaculos
16 MARGIN_CELLS = 12 # folga do bounding box em celulas (espaco nao observado)
17
18
19 def _cell_of(x, y, r):
20     return (int(math.floor(x / r)), int(math.floor(y / r)))
21
22
23 def _center(i, j, r):
24     return ((i + 0.5) * r, (j + 0.5) * r)
25
26
27 def _blocked_set(grid):
28     """Conjunto de celulas (i,j) bloqueadas = occupied dilatado pelo raio."""
29     r = float(grid.get("resolution_mm", 50.0))
30     occ = set()
31     for c in grid.get("cells", []):
32         if c.get("state") == "occupied":
33             occ.add((int(c["i"]), int(c["j"])))
34     if not occ:
35         return set(), r
36     infl = int(math.ceil(ROBOT_RADIUS_MM / r))
37     blocked = set()
38     for (i, j) in occ:
39         for di in range(-infl, infl + 1):
40             for dj in range(-infl, infl + 1):
41                 if di * di + dj * dj <= infl * infl:
42                     blocked.add((i + di, j + dj))
43     return blocked, r
44
45
46 def plan_waypoints(grid, start_xy, goal_xy):
47     """A* 8-conexo de start ate goal (mm). Retorna lista de waypoints (x,y) em mm,
48     ja simplificada por linha-de-visada. Retorna [] se nao houver caminho."""
49     blocked, r = _blocked_set(grid)
50     s = _cell_of(start_xy[0], start_xy[1], r)
51     g = _cell_of(goal_xy[0], goal_xy[1], r)
52     if s == g:
53         return [tuple(map(float, goal_xy))]
54
55     # bounding box: envolve start, goal e celulas observadas, com folga
56     xs = [s[0], g[0]]
57     ys = [s[1], g[1]]
58     for c in grid.get("cells", []):

```

```

59     xs.append(int(c["i"])); ys.append(int(c["j"]))
60     imin, imax = min(xs) - MARGIN_CELLS, max(xs) + MARGIN_CELLS
61     jmin, jmax = min(ys) - MARGIN_CELLS, max(ys) + MARGIN_CELLS
62
63     def inb(i, j):
64         return imin <= i <= imax and jmin <= j <= jmax
65
66     # o robo esta fisicamente na footprint do start: libera um disco em volta
67     # dele (senao um obstaculo mapeado colado na origem o deixa "preso").
68     infl = int(math.ceil(ROBOT_RADIUS_MM / r))
69     free_start = {(s[0] + di, s[1] + dj)
70                  for di in range(-infl, infl + 1)
71                  for dj in range(-infl, infl + 1)
72                  if di * di + dj * dj <= infl * infl}
73     blocked = blocked - free_start
74
75     neigh = [(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)]
76
77     def h(a, b):
78         return math.hypot(a[0] - b[0], a[1] - b[1])
79
80     def can_step(cur, di, dj):
81         nb = (cur[0] + di, cur[1] + dj)
82         if not inb(*nb) or nb in blocked:
83             return False
84         # Nao corta quina: em diagonal, os dois vizinhos ortogonais precisam
85         # estar livres para a footprint dilatada do robo passar com folga.
86         if di and dj and ((cur[0] + di, cur[1]) in blocked or
87                          (cur[0], cur[1] + dj) in blocked):
88             return False
89         return True
90
91     openh = [(h(s, g), 0.0, s)]
92     came = {}
93     gscore = {s: 0.0}
94     closed = set()
95     while openh:
96         _, gc, cur = heapq.heappop(openh)
97         if cur == g:
98             break
99         if cur in closed:
100             continue
101         closed.add(cur)
102         for di, dj in neigh:
103             nb = (cur[0] + di, cur[1] + dj)
104             if nb in closed or not can_step(cur, di, dj):
105                 continue
106             step = math.hypot(di, dj)
107             ng = gc + step
108             if ng < gscore.get(nb, float("inf")):
109                 gscore[nb] = ng
110                 came[nb] = cur
111                 heapq.heappush(openh, (ng + h(nb, g), ng, nb))
112     if g not in came and g != s:
113         return []
114
115     # reconstroi
116     path = [g]
117     while path[-1] != s:
118         path.append(came[path[-1]])
119     path.reverse()
120
121     # simplifica por linha-de-visada (Bresenham entre celulas)

```

```

122 def visible(a, b):
123     (x0, y0), (x1, y1) = a, b
124     dx, dy = abs(x1 - x0), abs(y1 - y0)
125     sx = 1 if x0 < x1 else -1
126     sy = 1 if y0 < y1 else -1
127     err = dx - dy
128     x, y = x0, y0
129     while True:
130         if (x, y) in blocked:
131             return False
132         if (x, y) == (x1, y1):
133             return True
134         px, py = x, y
135         e2 = 2 * err
136         if e2 > -dy:
137             err -= dy; x += sx
138         if e2 < dx:
139             err += dx; y += sy
140         if not can_step((px, py), x - px, y - py):
141             return False
142
143     simp = [path[0]]
144     k = 0
145     while k < len(path) - 1:
146         j = len(path) - 1
147         while j > k + 1 and not visible(path[k], path[j]):
148             j -= 1
149         simp.append(path[j])
150         k = j
151
152     # células -> waypoints em mm (centro da célula); ultimo = goal exato
153     wps = [_center(i, j, r) for (i, j) in simp]
154     wps[-1] = (float(goal_xy[0]), float(goal_xy[1]))
155     if len(wps) >= 2 and math.hypot(wps[0][0] - start_xy[0], wps[0][1] - start_xy[1]) < r:
156         wps = wps[1:] # descarta 1o waypoint coincidente com a pose atual
157     return wps

```

E Demais componentes do repositório

Por concisão, não se reproduzem aqui, na íntegra, os componentes de *ferramental* do projeto, disponíveis no repositório: o simulador visual OpenGL e o gerador da bateria de 1000 mundos (tools/sim_khepera/khepera_gl.c, generate_worlds.py, worlds_1000.json); a interface web de operação (interface/app.py, interface/static/index.html); o mundo do Webots (worlds/first.wbt); e os documentos de engenharia em L^AT_EX (entregaveis/: plano de projeto, análise de requisitos, arquitetura, plano e protocolo de testes, código-fonte e apresentação final).